Language Models for Law and Social Science

6. Sequences and Attention

Social Science with Embeddings

Social Science with Embeddings

- What are we measuring?
  - e.g., how do measurements by person or by group correlate with actual attitudes, for example as measured by IAT scores?

Social Science with Embeddings

- What are we measuring?
  - e.g., how do measurements by person or by group correlate with actual attitudes, for example as measured by IAT scores?
- What attitudes can be reliably measured?
  - e.g. racial sentiment, especially in light of black sheep problem.

Social Science with Embeddings

- What are we measuring?
  - e.g., how do measurements by person or by group correlate with actual attitudes, for example as measured by IAT scores?
- What attitudes can be reliably measured?
  - e.g. racial sentiment, especially in light of black sheep problem.
- In what domains is this relevant?
  - social media, news media, politics, legal, scientific, ...

Social Science with Embeddings

- What are we measuring?
  - e.g., how do measurements by person or by group correlate with actual attitudes, for example as measured by IAT scores?
- What attitudes can be reliably measured?
  - e.g. racial sentiment, especially in light of black sheep problem.
- In what domains is this relevant?
  - social media, news media, politics, legal, scientific, ...
- Does language matter?
  - Djourelova (2020): style change from "illegal" to "undocumented" immigrant softened attitudes toward immigration.

Examples: Confounders in Measurement from Text

What quantity do we care about? vs. What do we measure?

Examples: Confounders in Measurement from Text

What quantity do we care about? vs. What do we measure?

- Positive/negative sentiment $\rightarrow$ Count positive/negative words, or predict text annotations.
- Toxicity $\rightarrow$ Count toxic words, or predict text annotations.

confounders?

Examples: Confounders in Measurement from Text

What quantity do we care about? vs. What do we measure?

- Positive/negative sentiment $\rightarrow$ Count positive/negative words, or predict text annotations.
- Toxicity $\rightarrow$ Count toxic words, or predict text annotations.

confounders?

- Student performance $\rightarrow$ predicted essay grade based on labeled essay documents.
- Credit worthiness $\rightarrow$ predicted probability of default based on loan application documents.

confounders?

Examples: Confounders in Measurement from Text

What quantity do we care about? vs. What do we measure?

- Positive/negative sentiment $\rightarrow$ Count positive/negative words, or predict text annotations.
- Toxicity $\rightarrow$ Count toxic words, or predict text annotations.

confounders?

- Student performance $\rightarrow$ predicted essay grade based on labeled essay documents.
- Credit worthiness $\rightarrow$ predicted probability of default based on loan application documents.

confounders?

- Political partisanship $\rightarrow$ predicted probability being Democrat/Republican based on speeches.

confounders?

Examples: Confounders in Measurement from Text

What quantity do we care about? vs. What do we measure?

- Positive/negative sentiment → Count positive/negative words, or predict text annotations.
- Toxicity → Count toxic words, or predict text annotations.

confounders?

- Student performance → predicted essay grade based on labeled essay documents.
- Credit worthiness → predicted probability of default based on loan application documents.

confounders?

- Political partisanship → predicted probability being Democrat/Republican based on speeches.

confounders?

- Policy priorities → predicted probability of speeches/laws being about a particular policy topic.

confounders?

When is measurement confounding important?

- By itself, producing measurements that are biased by confounders might not be a problem.
- e.g.:
  - an NLP-based essay grading system that learns confounders $\rightarrow$ not a problem unless students learn about it and strategically alter their essays.

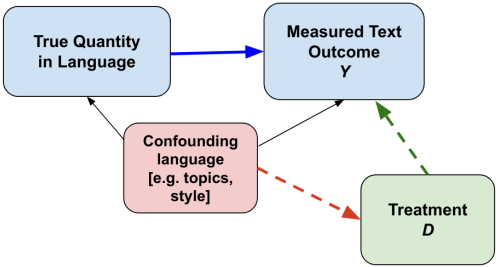
When is measurement confounding important?

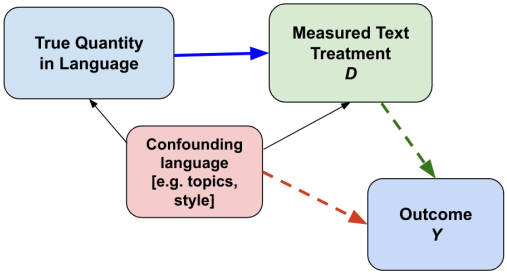
- By itself, producing measurements that are biased by confounders might not be a problem.
- e.g.:
  - an NLP-based essay grading system that learns confounders $\rightarrow$ not a problem unless students learn about it and strategically alter their essays.
- for measuring political divisiveness or policy priorities
  - probably won't matter for in-domain summary statistics
  - but would matter a lot for summary statistics in a new domain

When is measurement confounding important?

- By itself, producing measurements that are biased by confounders might not be a problem.
- e.g.:
  - an NLP-based essay grading system that learns confounders $\rightarrow$ not a problem unless students learn about it and strategically alter their essays.
- for measuring political divisiveness or policy priorities
  - probably won't matter for in-domain summary statistics
  - but would matter a lot for summary statistics in a new domain
- even in-domain, will matter for assessing the causal effect of a treatment, e.g. the electoral cycle:
  - elections might cause politicians to focus on social issues rather than economic issues,
  - if social/economic issues are confounded with partisanship, the resulting estimates are biased.

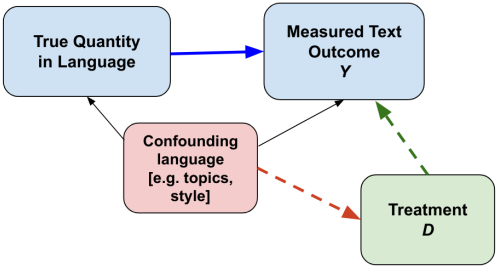
When is measurement confounding important?

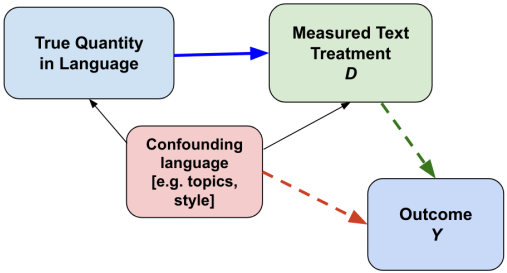




- When text is outcome, the confounders cannot be correlated with the treatment.
- When text is treatment, the confounders cannot be correlated with the outcome.

When is measurement confounding important?





- When text is outcome, the confounders cannot be correlated with the treatment.
- When text is treatment, the confounders cannot be correlated with the outcome.
  - e.g.: estimating the effect of politician speech sentiment on his/her reelection chances?

What is (Document) Embedding?

"Embedding": a lower-dimensional dense vector representation of a higher-dimensional object

- also refers to algorithm for making such vectors

What is (Document) Embedding?

"Embedding": a lower-dimensional dense vector representation of a higher-dimensional object

- also refers to algorithm for making such vectors

Document vectors:

- quantitative analysis of language requires that documents be transformed to numbers – that is, vectors.

What is (Document) Embedding?

"Embedding": a lower-dimensional dense vector representation of a higher-dimensional object

- also refers to algorithm for making such vectors

Document vectors:

- quantitative analysis of language requires that documents be transformed to numbers – that is, vectors.
- standard approach:
  - represent documents as sparse vectors of token counts/frequencies.
  - e.g., to compute document similarity measures, to learn topics, or to use as features in supervised learning

What is (Document) Embedding?

"Embedding": a lower-dimensional dense vector representation of a higher-dimensional object

- also refers to algorithm for making such vectors

Document vectors:

- quantitative analysis of language requires that documents be transformed to numbers – that is, vectors.
- standard approach:
  - represent documents as sparse vectors of token counts/frequencies.
  - e.g., to compute document similarity measures, to learn topics, or to use as features in supervised learning
- **Embedding approach**:
  - low-dimensional dense vectors rather than high-dimensional sparse vectors
  - Embedding without neural nets:
    - PCA reductions of the document-term matrix
    - LDA topic shares

What is (Document) Embedding?

"Embedding": a lower-dimensional dense vector representation of a higher-dimensional object

- also refers to algorithm for making such vectors

Document vectors:

- quantitative analysis of language requires that documents be transformed to numbers – that is, vectors.
- standard approach:
  - represent documents as sparse vectors of token counts/frequencies.
  - e.g., to compute document similarity measures, to learn topics, or to use as features in supervised learning
- **Embedding approach**:
  - low-dimensional dense vectors rather than high-dimensional sparse vectors
  - Embedding without neural nets:
    - PCA reductions of the document-term matrix
    - LDA topic shares
  - Embedding with neural nets:
    - many useful ways to do this.

Outline

Embedding Sequences Without Word Order (CBOW)

The C-SPAN Effect

Embedding Sequences without Transformers

Transformers: Embedding Sequences with Attention

Word Vectors can produce Document Vectors

$$\vec{D} = \sum_{w \in D} a_w \vec{w}$$

- The "continuous bag of words" (CBOW) representation for document D is the sum, or the average (potentially weighted by a_w), of the vectors $\vec{w}$ for each word w in the document.
  - word vectors $\vec{w}$ constructed using pre-trained GloVe or Word2Vec.
  - "Document" could be sentence, paragraph, section, etc. (scales well to long docs)

Word Vectors can produce Document Vectors

$$\vec{D} = \sum_{w \in D} a_w \vec{w}$$

- The "continuous bag of words" (CBOW) representation for document D is the sum, or the average (potentially weighted by a_w), of the vectors $\vec{w}$ for each word w in the document.
  - word vectors $\vec{w}$ constructed using pre-trained GloVe or Word2Vec.
  - "Document" could be sentence, paragraph, section, etc. (scales well to long docs)
- Arora, Liang, and Ma (2017) provide a "tough to beat baseline", the SIF-weighted ("smoothed inverse frequency") average of the vectors:

$$a_w = \frac{\alpha}{\alpha + p_w}$$

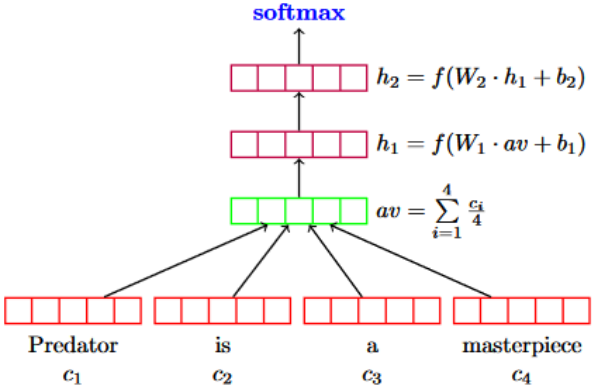
where p_w is the probability (frequency) of the word and $\alpha = .001$ is a smoothing parameter.

Deep Averaging Network (Iyyer et al 2015)

- Similar to averaged word embeddings across doc, but embeddings are learned during neural net training:

Deep Averaging Network (Iyyer et al 2015)

- Similar to averaged word embeddings across doc, but embeddings are learned during neural net training:



1. Trainable embedding layer for words, initialized with pre-trained embeddings
2. Average the embeddings, with dropout (sometimes words left out of average)
3. Average embedding fed into MLP with multiple hidden layers
4. MLP outputs used for classification or regression

Hashed N-Gram Embeddings (Joulin et al 2016)

Combine the Iyyer et al (2015) approach with the hashing n-gram vectorizer.

Hashed N-Gram Embeddings (Joulin et al 2016)

Combine the Iyyer et al (2015) approach with the hashing n-gram vectorizer.

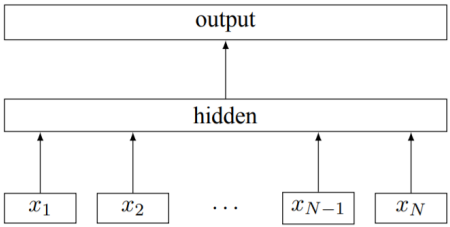


Figure 1: Model architecture of `fastText` for a sentence with N ngram features $x_1, \ldots, x_N$. The features are embedded and averaged to form the hidden variable.

1. Allocate $n_w \approx 10$ million rows to embedding matrix.
2. Assign n-grams to embedding indexes with hashing function.
3. sentence embedding = average of n-gram embeddings
4. send to dense hidden layer(s)
5. send to output (e.g. classifier / regressor).

- Captures local word-order information from n-grams without building vocabulary or costly training of Convolutional Neural Net.

Outline

Embedding Sequences Without Word Order (CBOW)

The C-SPAN Effect

Embedding Sequences without Transformers

Transformers: Embedding Sequences with Attention

Measuring Emotionality from Text

Gennaro, Ash (2022)
Emotion and reason in political language
The Economic Journal

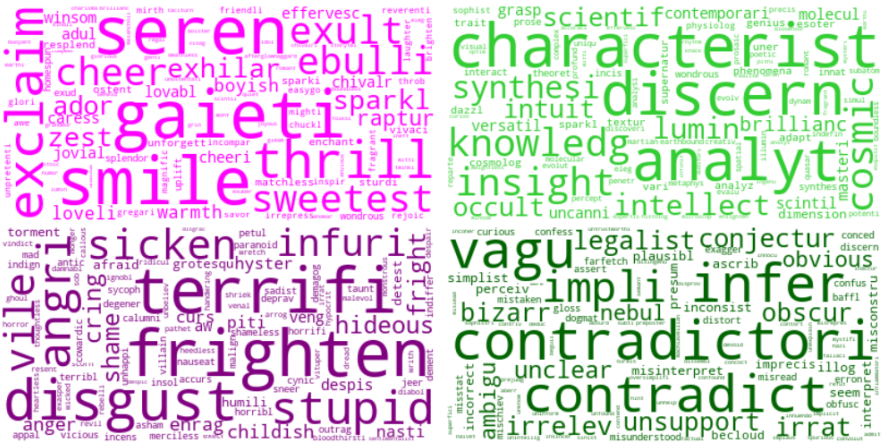
`github.com/elliottash/emotionmeter`

Congressional Speeches Corpus

- Universe of floor speeches in U.S. Congress from *Congressional Record*, 1858-2014 (excluding non-speech content, N=9 799 375)
- We train a word-embedding model to obtain a vector representation of political language **Intuition**: words that occur in similar contexts are semantically related/similar (and obtain similar word vectors)
- Seed dictionaries of emotional/logical words (Tausczik and Pennebaker, 2010): "Cognitive Processing" (causation, comparison, uncertainty, inclusion, exclusion) vs. "Affective Processing" (positive and negative emotions/moods)
- Each dictionary is represented by its centroid in the word embedding space
- **Measure** of relative closeness to emotion vs. reason centroids Details

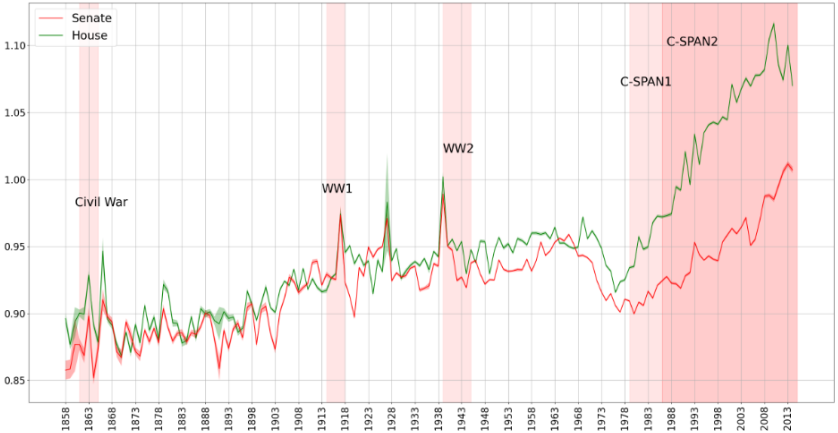
Words that are Distinctive of the Emotion and Reason Dimensions

Gennaro, Ash (2022), Emotion and Reason in Political Language, *The Economic Journal*


serenity
exclaim
gaieti
smile
thrill
sweetest
exult
ebulli
cheer
exhilar
sparkl
boyish
chivalr
ador
zest
warmth
lovell
effervesc
winsom
adul
cesplend
caress
jovial
cheeri
enchant
unforgett
incompar
matchless
inspir
sturdi
wondrous
rejoic
irrepress
savor
gregari
friendli
reverenti
mirth
sicken
infuri
terrifi
frighten
disgust
stupid
angri
vile
torment
afraid
grotesqu
hyster
petul
paranoid
wretch
sycoph
degener
calumni
curs
aw
piti
shameless
horrifi
hideous
shame
cring
detest
fright
indignat
terribl
insol
accurs
despis
cynic
sneer
humili
horribl
childish
nasti
anger
appal
vicious
incens
merciless
asham
enrag
outrag
bloodthirsti
taunt
malevol
venal
veng
sadist
deprav
arrog
shriek
hauseat
needlest
villain
pathet
wrath
malign
scamp
resent
resell
swapp
insolv
revil
dubiot
jeer
scientif
contemporari
molecul
grasp
physiolog
genius
geoter
characterist
discern
synthesi
intuit
versatil
knowledg
lumin
brillianc
analyt
cosmic
insight
occult
uncanni
intellect
scintil
dimension
masteri
vagu
legalist
conjectur
obvious
impli
infer
obscur
bizarr
nebul
perceiv
mistaken
simplist
farfetch
plausibl
ascrib
assert
presum
inconsist
distort
contradictori
contradict
unclear
misinterpret
misread
imprecis
illog
irrelev
unsupport
irrat
becloud
misunderstood
ambigu
incorrect
misstat
misconstru
interpret
seem
obfusc
confus
baffl
conced
discern
curious
confess
unfortunatelyy
sophist
trait
prose
visual
uniqu
interact
theoret
incis
supernatur
wondrous
phenomena
innat
subatom
evolv
dynam
dazzl
textur
sparkl
adapt
cosmolog
molecular
evolut
elog
metaphys
analyz
synthes
percept
genetic
vari
uniquely
uniquelyy
uniquelyyy
uniquelyyyy
uniquelyyyyy
uniquelyyyyyy
uniquelyyyyyyy
uniquelyyyyyyyy
uniquelyyyyyyyyy
uniquelyyyyyyyyyy
uniquelyyyyyyyyyyy
uniquelyyyyyyyyyyyy
uniquelyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquelyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
uniquely

Emotionality by Chamber over Time

Gennaro, Ash (2022), Emotion and Reason in Political Language, *The Economic Journal*



C-SPAN and Emotionality

How Introducing C-SPAN Affected Emotional Rhetoric

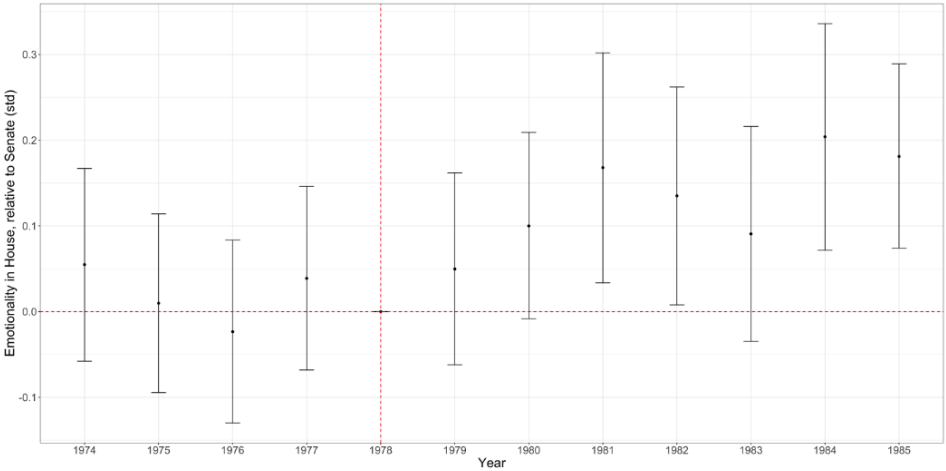
Panel Event Study Approach

- Staggered introduction of C-SPAN in the House (1979) and in the Senate (1986)

$$Y_{ijt} = \alpha + \sum_{m=-5}^{6} \beta_m H_j \times L_{1979+m} + H_j + \tau_t + \epsilon_{ijt}$$

- Y_{ijt} is emotionality in speech i, chamber j, year t
- H_j is a dummy for the House
- L_{1979+m} are dummies for leads and lags around 1979
- τ_t are year fixed effects
- Standard errors are clustered by speaker

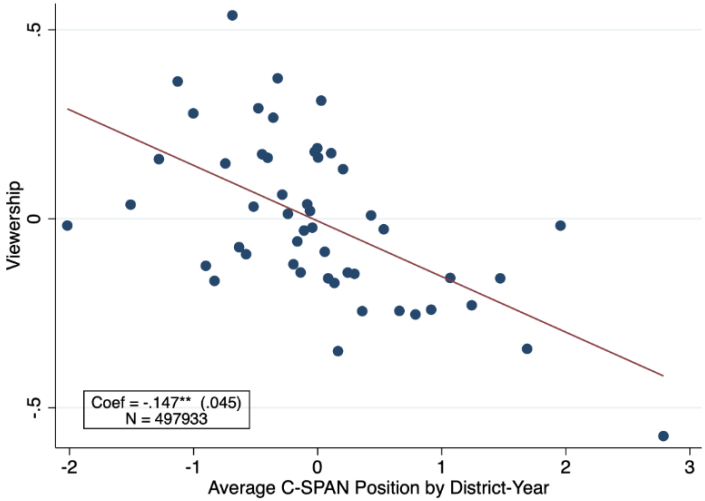
Emotionality in House, Relative to Senate, 1974-1985



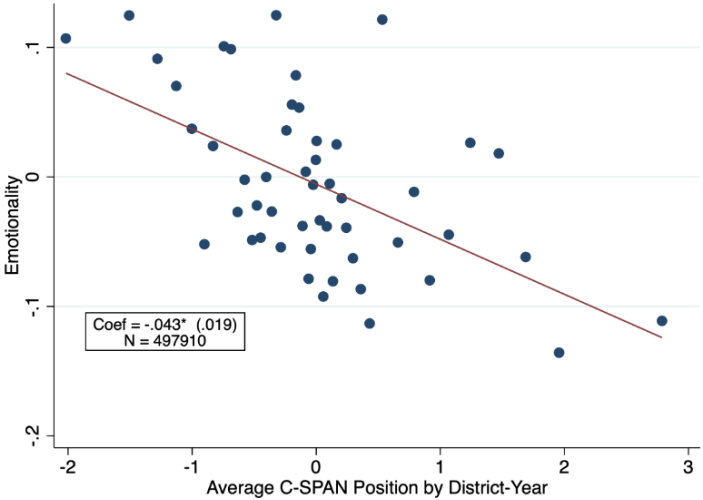
C-SPAN Ratings and Channel Position

- We estimate effect of district-level C-SPAN viewership on the Reps' use of emotionality
  - To corroborate the DiD effect of C-SPAN Viewership on Emotionality
  - Less vulnerable to confounding policy changes
- Key identification challenge: home district characteristics are likely to affect both C-SPAN viewership and MCs
- Instrument for local viewership is C-SPAN Channel Position (Martin and Yurukoglu, 2017)
  - Viewers tend to start at the bottom of the lineup and surf upward
  - Position depends on when the media producer joined the local cable (and drop-outs)
  - The positioning rarely changes over time

First Stage


Viewership
.5
0
-.5
Coef = -.147** (.045)
N = 497933
-2
-1
0
1
2
3
Average C-SPAN Position by District-Year

Main Results: Reduced Form



Robustness

- Controls
  - Exogenous controls improve precision
  - Mediators shrink estimate magnitude but it remains positive and significant
  - Instrument balance Table
- Placebos
  - No effect of C-SPAN on emotionality in the two decades before introduction Table
  - No effect of C-SPAN2 on emotionality in the House Table
- Specification checks
  - Analysis at the district-year level Figures
  - Inverse hyperbolic sine transformation Figures
  - Leave one State or Year out Figures
  - Text-based emotionality metric is significantly correlated with Dietrich et al's (2019) audio-based emotionality metric

Outline

Embedding Sequences Without Word Order (CBOW)

The C-SPAN Effect

Embedding Sequences without Transformers

Transformers: Embedding Sequences with Attention

Addressing High Dimensionality in Text (Ash & Hansen 2023)

- Dictionaries: count the terms you care about (Baker et al 2016; Hassan et al 2019; Enke 2020)
  $\rightarrow$ dimensionality is the number of tagged categories from the dictionary.

Addressing High Dimensionality in Text (Ash & Hansen 2023)

- Dictionaries: count the terms you care about (Baker et al 2016; Hassan et al 2019; Enke 2020)
  $\rightarrow$ dimensionality is the number of tagged categories from the dictionary.

- Bag-of-words models: count each word/phrase in a vocabulary (Hoberg & Phillips 2016, Cagé et al 2020; Kelly et al 2021)
  $\rightarrow$ dimensionality is the vocabulary size.

Addressing High Dimensionality in Text (Ash & Hansen 2023)

- Dictionaries: count the terms you care about (Baker et al 2016; Hassan et al 2019; Enke 2020)
  $\rightarrow$ dimensionality is the number of tagged categories from the dictionary.

- Bag-of-words models: count each word/phrase in a vocabulary (Hoberg & Phillips 2016, Cagé et al 2020; Kelly et al 2021)
  $\rightarrow$ dimensionality is the vocabulary size.

- Factor models: apply PCA or LDA to the bag-of-words representation (Hansen et al 2018; Bertrand et al 2021)
  $\rightarrow$ dimensionality is the number of low-dimensional factors.

Addressing High Dimensionality in Text (Ash & Hansen 2023)

- Dictionaries: count the terms you care about (Baker et al 2016; Hassan et al 2019; Enke 2020)
  $\rightarrow$ dimensionality is the number of tagged categories from the dictionary.

- Bag-of-words models: count each word/phrase in a vocabulary (Hoberg & Phillips 2016, Cagé et al 2020; Kelly et al 2021)
  $\rightarrow$ dimensionality is the vocabulary size.

- Factor models: apply PCA or LDA to the bag-of-words representation (Hansen et al 2018; Bertrand et al 2021)
  $\rightarrow$ dimensionality is the number of low-dimensional factors.

- Static word embeddings: Vectorize documents as averaged word embeddings (Gennaro and Ash 2022; Ash et al 2024)
  $\rightarrow$ dimensionality is determined by the embeddings.

Addressing High Dimensionality in Text (Ash & Hansen 2023)

- Dictionaries: count the terms you care about (Baker et al 2016; Hassan et al 2019; Enke 2020)
  $\rightarrow$ dimensionality is the number of tagged categories from the dictionary.

- Bag-of-words models: count each word/phrase in a vocabulary (Hoberg & Phillips 2016, Cagé et al 2020; Kelly et al 2021)
  $\rightarrow$ dimensionality is the vocabulary size.

- Factor models: apply PCA or LDA to the bag-of-words representation (Hansen et al 2018; Bertrand et al 2021)
  $\rightarrow$ dimensionality is the number of low-dimensional factors.

- Static word embeddings: Vectorize documents as averaged word embeddings (Gennaro and Ash 2022; Ash et al 2024)
  $\rightarrow$ dimensionality is determined by the embeddings.

- Sequence models:
  $\rightarrow$

Why Sequence Models are Needed

- Synonymy:
  - the legal ruling was **rejected**
  - the legal ruling was **invalidated**
- Polysemy
  - The court **stayed** the proceedings.
  - The defendant **stayed** in the jurisdiction.
- Sequence
  - Congress passed **limits** on the budget and **expansions** on taxes.
  - Congress passed **expansions** on the budget and **limits** on taxes.

Sequence Models

- The real break-through in deep learning for NLP:
  - moving from bag-of-X representations to **sequence representations**.
  - Rather than processing counts (or topics/embeddings), process the **sequence of words – i.e., tokens** $\{w_1, \ldots, w_t, \ldots w_n\}$.

Sequence Models

- The real break-through in deep learning for NLP:
  - moving from bag-of-X representations to **sequence representations**.
  - Rather than processing counts (or topics/embeddings), process the **sequence of words – i.e., tokens** $\{w_1, \ldots, w_t, \ldots w_n\}$.

- We can't read sequences directly due to high dimensionality and sparsity.
- Neural networks, and in particular transformers, solve this problem by learning informative, context-sensitive embeddings that capture the full meaning of the document.

From last time: Embedding every token in a document

- **Embedding layers** take a categorical variable as input and produce a low-dimensional dense representation.

From last time: Embedding every token in a document

- **Embedding layers** take a categorical variable as input and produce a low-dimensional dense representation.

We saw this example last time, which produces document embeddings:

- Tokenize document to fixed length n_L
- Inputs are each word position, input categorical (word) to n_E-dimensional embedding layer:

$$\boldsymbol{x}_{1:n_L} = [\ \boldsymbol{x}_1 \quad \ldots \quad \boldsymbol{x}_t \quad \ldots \quad \boldsymbol{x}_{n_L}\ ]$$

- pipe to further hidden layers of network.

From last time: Embedding every token in a document

- **Embedding layers** take a categorical variable as input and produce a low-dimensional dense representation.

We saw this example last time, which produces document embeddings:

- Tokenize document to fixed length n_L
- Inputs are each word position, input categorical (word) to n_E-dimensional embedding layer:

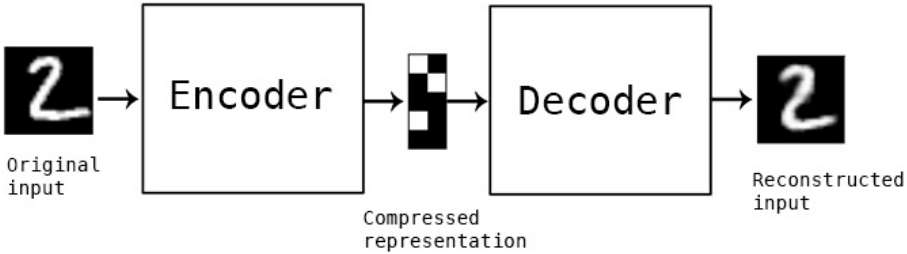
$$\boldsymbol{x}_{1:n_L} = [\ \boldsymbol{x}_1 \quad \ldots \quad \boldsymbol{x}_t \quad \ldots \quad \boldsymbol{x}_{n_L}\ ]$$

- pipe to further hidden layers of network.
- document embedding $= n_L n_E$-dimensional vector of concatenated word embeddings.
  - computationally demanding and only works with short documents.

Autoencoders: Optimal Compression Algorithms

Autoencoders: Optimal Compression Algorithms

- Autoencoders = neural nets that perform domain-specific lossy compression:



- Learned encodings can be decoded back to a *reconstruction* – a (minimally) lossy representation of the original data.
- AE's can memorize complex, unstructured data – deep unsupervised learning.

Autoencoder Architecture: Neural net with output=input

- Stacked layers gradually decrease in dimensionality to create the compressed representation
- then gradually increase in dimensionality to try to reconstruct the input.

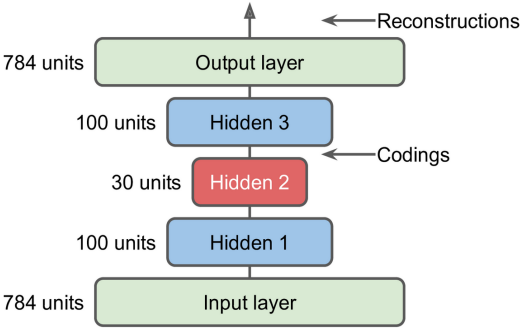


Figure 17-3. Stacked autoencoder

Reconstruction from encoded vector

Figure 17-4. Original images (top) and their reconstructions (bottom)

Autoencoder Encodings are Embeddings

- Autoencoder compresses a document (e.g. a sentence) into a vector to be reconstructed.
  - Can use the compressed representation as a document embedding.
- Standard (that is, non-transformer) autoencoder embeddings don't tend to work well for sentence similarity tasks because autoencoders try to reproduce the specific wording (reconstruction objective), rather than the semantic meaning.
  - transformer-based autoencoders, i.e. BART, address this issue

Convolutional Neural Nets ↔ N-gram Detectors

- A neural net architecture that constructs **filters** that slide across input sequences and extract **local predictive structure**.

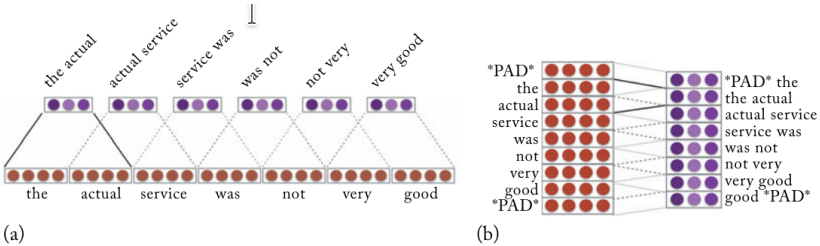


Figure 13.1: The inputs and outputs of a narrow and a wide convolution in the vector-concatenation and the vector-stacking notations. (a) A *narrow* convolution with a window of size $k = 2$ and 3-dimensional output ($\ell = 3$), in the vector-concatenation notation. (b) A *wide* convolution with a window of size $k = 2$, a 3-dimensional output ($\ell = 3$), in the vector-stacking notation.

- Overall, CNNs do not work well in NLP; use embedded hashed n-grams instead (Joulin et al 2016, Goldberg 2017).

RNNs can input and output arbitrary-length sequences

- Downsides of previous approaches:
  - CBOW models (averaged word/phrase embeddings) lose any sequence information beyond local word order encoded by n-grams
  - all-token embedding, and CNNs, require fixed-length documents

RNNs can input and output arbitrary-length sequences

- Downsides of previous approaches:
  - CBOW models (averaged word/phrase embeddings) lose any sequence information beyond local word order encoded by n-grams
  - all-token embedding, and CNNs, require fixed-length documents
- Recurrent Neural Nets (RNNs) work with **sequences of arbitrary length**, both as **inputs** and **outputs**:
  - can *encode* sequences into vectors.
  - can *decode* vectors into sequences.
- therefore especially useful for language tasks such as translation.

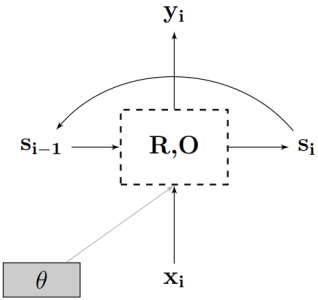
Summary of RNN Architecture

- At each step t:
  - a recursion function $R(s_{t-1}, x_t; \theta_R)$ computes the state vector s_t given current word x_t and previous state s_{t-1}.

Summary of RNN Architecture

- At each step t:
  - a recursion function $R(s_{t-1}, x_t; \theta_R)$ computes the state vector s_t given current word x_t and previous state s_{t-1}.
  - An output function $O(s_t; \theta_O)$ computes an output vector y_t (to be compared to the outcome variable in the dataset).

$$\hat{\boldsymbol{y}}_t = O(\boldsymbol{s}_t, \theta_O)$$
$$\boldsymbol{s}_t = R(\boldsymbol{s}_{t-1}, \boldsymbol{x}_t, \theta_R)$$



- The parameters of those functions, $\theta = (\theta_R, \theta_O)$ are learned during model training.

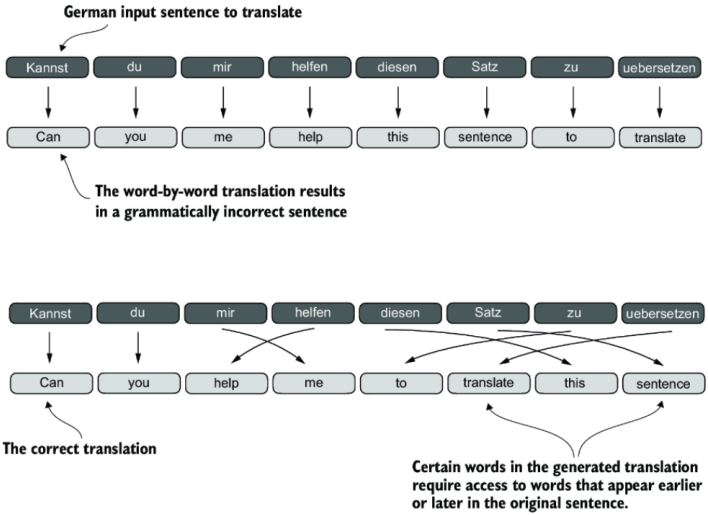


Figure 3.3 When translating text from one language to another, such as German to English, it's not possible to merely translate word by word. Instead, the translation process requires contextual understanding and grammatical alignment.

RNN Encoding and Decoding

top left: sequence to sequence; top right: sequence to vector

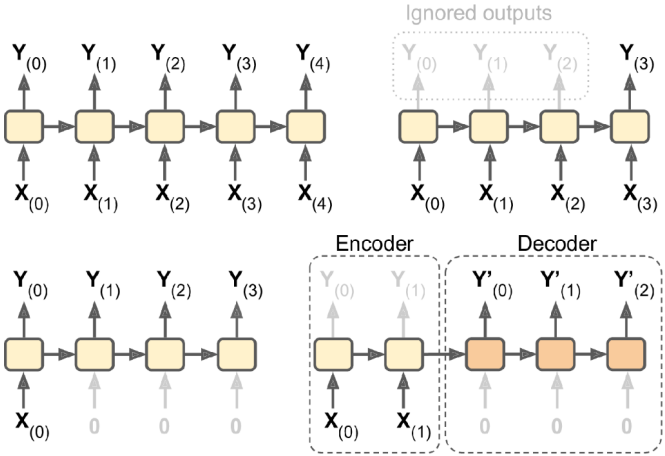


Figure 15-4. Seq-to-seq (top left), seq-to-vector (top right), vector-to-seq (bottom left), and Encoder–Decoder (bottom right) networks

bottom left: vector to sequence; bottom right: encoder-decoder.

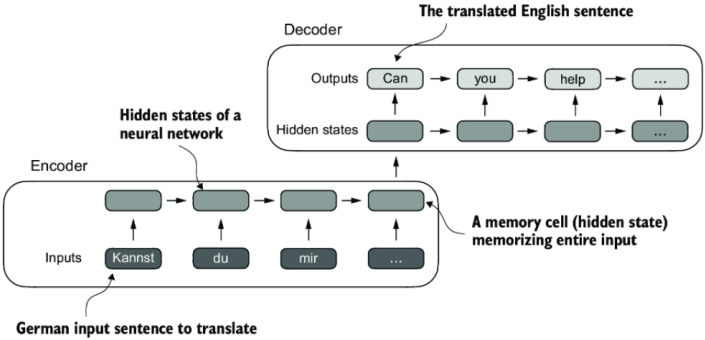
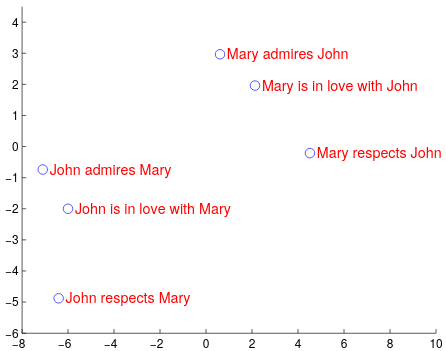
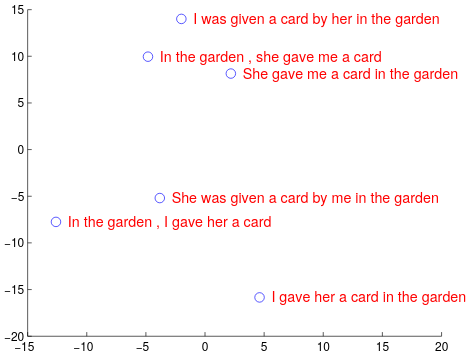


Figure 3.4 Before the advent of transformer models, encoder–decoder RNNs were a popular choice for machine translation. The encoder takes a sequence of tokens from the source language as input, where a hidden state (an intermediate neural network layer) of the encoder encodes a compressed representation of the entire input sequence. Then, the decoder uses its current hidden state to begin the translation, token by token.

RNN's (e.g. Machine Translation) Produce Document Embeddings

- RNN machine translators produce a sentence vector that must be decoded into another language.
- if the vector produces a good translation, it must contain the important information in the sentence.





Sutskever, Vinyals, and Le, "Sequence to sequence learning with neural networks."

Outline

Embedding Sequences Without Word Order (CBOW)

The C-SPAN Effect

Embedding Sequences without Transformers

Transformers: Embedding Sequences with Attention

Deep Learning with NLP $\approx$ Transformers

- Since a 2017 paper (Vaswani et al 2017), most deep learning for NLP uses the transformer architecture.
- Recurrent neural nets can process whole documents, but they have to sweep through the whole document word-by-word.
  - recursion mechanism is heavily biased toward local/nearby context.
  - and RNNs learn too slowly.
  - but this is an active area of research: e.g. Mamba (Gu and Dao 2024), Griffin (De et al 2024).
- Transformers overcome this limitation:
  - intuitively, transforms provide a way to efficiently read in an entire document and learn the meaning of all words and all interactions between words.
  - even at a long distance (unlike n-grams / RNNs which are biased toward local context).

Transformers = Attentional Neural Nets

- Attention refers to a neural-net architecture/layer, like dense layers, embedding layers, convolutions, or recurrence mechanisms.
  - introduced by Bahdanau et al (2014) as an addition to recurrent neural nets for the translation task.

Transformers = Attentional Neural Nets

- Attention refers to a neural-net architecture/layer, like dense layers, embedding layers, convolutions, or recurrence mechanisms.
  - introduced by Bahdanau et al (2014) as an addition to recurrent neural nets for the translation task.

Attention does two things:

1. On a technical level:
   - allows a neural net to build many implicit databases of key-value pairs (a la python dictionaries), and to efficiently query those databases.

Transformers = Attentional Neural Nets

- Attention refers to a neural-net architecture/layer, like dense layers, embedding layers, convolutions, or recurrence mechanisms.
  - introduced by Bahdanau et al (2014) as an addition to recurrent neural nets for the translation task.

Attention does two things:

1. On a technical level:
   - allows a neural net to build many implicit databases of key-value pairs (a la python dictionaries), and to efficiently query those databases.
2. On a linguistic level:
   - allows a neural net to build a set of implicit key-value databases:
     - the query-keys are pairs of words
     - the value is a learnable vector that helps in some prediction task, e.g. predicting the next word in a sequence.

https://x.com/karpathy/status/1864023344435380613

Andrej Karpathy
@karpathy

Attention is a brilliant (data-dependent) weighted average operation. It is a form of global pooling, a reduction, communication. It is a way to aggregate relevant information from multiple nodes (tokens, image patches, or etc.). It is expressive, powerful, has plenty of parallelism, and is efficiently optimizable. Even the Multilayer Perceptron (MLP) can actually be almost re-written as Attention over data-indepedent weights (1st layer weights are the queries, 2nd layer weights are the values, the keys are just input, and softmax becomes elementwise, deleting the normalization). TLDR Attention is awesome and a *major* unlock in neural network architecture design.

Attention heads

- Transformers consist of stacked blocks of parallel **attention heads**
- **Attention heads** are machine-reading filters, which allow each word to scan over every other word in the document and pick up predictive interactions.

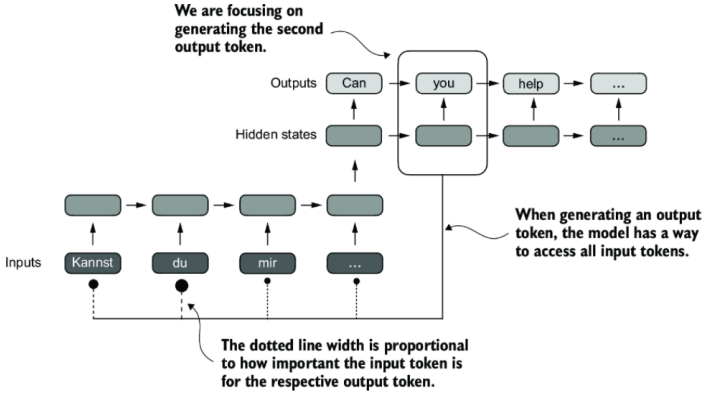


Figure 3.5 Using an attention mechanism, the text-generating decoder part of the network can access all input tokens selectively. This means that some input tokens are more important than others for generating a given output token. The importance is determined by the attention weights, which we will compute later. Note that this figure shows the general idea behind attention and does not depict the exact implementation of the Bahdanau mechanism, which is an RNN method outside this book's scope.

GPT and BERT are Pre-Trained Transformer Models

- **GPT = "Generative Pre-Trained Transformer"** – autoregressive / "decoder-only" model:
  - train transformer to predict the next word at the end of a sequence.
  - Three versions (GPT-1, GPT-2, GPT-3)
    - newer versions (GPT-3.5, GPT-4, GPT-4.5) add reinforcement learning with human alignment

GPT and BERT are Pre-Trained Transformer Models

- **GPT = "Generative Pre-Trained Transformer"** – autoregressive / "decoder-only" model:
  - train transformer to predict the next word at the end of a sequence.
  - Three versions (GPT-1, GPT-2, GPT-3)
    - newer versions (GPT-3.5, GPT-4, GPT-4.5) add reinforcement learning with human alignment
  - notably good for text generation (language modeling)

GPT and BERT are Pre-Trained Transformer Models

- **GPT = "Generative Pre-Trained Transformer"** – autoregressive / "decoder-only" model:
  - train transformer to predict the next word at the end of a sequence.
  - Three versions (GPT-1, GPT-2, GPT-3)
    - newer versions (GPT-3.5, GPT-4, GPT-4.5) add reinforcement learning with human alignment
  - notably good for text generation (language modeling)
- **BERT = "Bidrectional Encoder Representations from Transormers"** – bidirectional / "encoder-only" model:
  - train transformer to predict a single left-out word in the middle of a sequence.

GPT and BERT are Pre-Trained Transformer Models

- **GPT = "Generative Pre-Trained Transformer"** – autoregressive / "decoder-only" model:
  - train transformer to predict the next word at the end of a sequence.
  - Three versions (GPT-1, GPT-2, GPT-3)
    - newer versions (GPT-3.5, GPT-4, GPT-4.5) add reinforcement learning with human alignment
  - notably good for text generation (language modeling)
- **BERT = "Bidrectional Encoder Representations from Transormers"** – bidirectional / "encoder-only" model:
  - train transformer to predict a single left-out word in the middle of a sequence.
  - the resulting document embeddings contain most of the relevant information in short language snippets.
    - blew away all the NLP baselines (e.g. semantic role labeling, question-answering, entailment, etc.) when it came out in 2018.
    - further improved by RoBERTa (2019) and more recently ModernBERT (2024).

GPT and BERT are Pre-Trained Transformer Models

- **GPT = "Generative Pre-Trained Transformer"** – autoregressive / "decoder-only" model:
  - train transformer to predict the next word at the end of a sequence.
  - Three versions (GPT-1, GPT-2, GPT-3)
    - newer versions (GPT-3.5, GPT-4, GPT-4.5) add reinforcement learning with human alignment
  - notably good for text generation (language modeling)
- **BERT = "Bidrectional Encoder Representations from Transormers"** – bidirectional / "encoder-only" model:
  - train transformer to predict a single left-out word in the middle of a sequence.
  - the resulting document embeddings contain most of the relevant information in short language snippets.
    - blew away all the NLP baselines (e.g. semantic role labeling, question-answering, entailment, etc.) when it came out in 2018.
    - further improved by RoBERTa (2019) and more recently ModernBERT (2024).
  - immediately relevant use cases for our purpose:
    - straightforward to replace the single word-classification output layer with some other variable and fine-tune for a different prediction task.
    - many pre-trained models, e.g. for sentiment classification.

Shortcut: Using huggingface Pre-Trained Models

Shortcut: Using huggingface Pre-Trained Models

```
from transformers import pipeline
sentiment_analysis = pipeline(“sentiment-analysis”)

pos_text = "I enjoy studying computational algorithms."
neg_text = "I dislike sleeping late everyday."

pos_sent = sentiment_analysis(pos_text)[0]
print(pos_sent['label'],0 pos_sent['score'])

neg_sent = sentiment_analysis(neg_text)[0]
print(neg_sent['label'], neg_sent['score'])
```

- also straightforward to fine-tune BERT for your own classification tasks.
- see notebooks for full details / explanation.

Queries, Keys, and Values

- Assume a database $D = \{(k_1, v_1), ...(k_m, v_m)\}$
  - m tuples of keys and values.
  - Denote by q a "query".
  - e.g., a python dictionary; query is used to look-up; normally one of the keys.

Queries, Keys, and Values

- Assume a database $D = \{(k_1, v_1), \ldots (k_m, v_m)\}$
  - m tuples of keys and values.
  - Denote by q a "query".
  - e.g., a python dictionary; query is used to look-up; normally one of the keys.

Define

$$\text{Attention}(q) = \sum_{i=1}^{m} a(q, k_i) v_i$$

- $a(\cdot)$ are scalar "attention weights"; they give more weight ("pay more attention") to some items based on q and k_i.
- In a normal database query / dictionary, $a(q, k) = 1$ if $q = k$ and zero otherwise.

Queries, Keys, and Values

- Assume a database $D = \{(k_1, v_1), \ldots (k_m, v_m)\}$
  - m tuples of keys and values.
  - Denote by q a "query".
  - e.g., a python dictionary; query is used to look-up; normally one of the keys.

Define

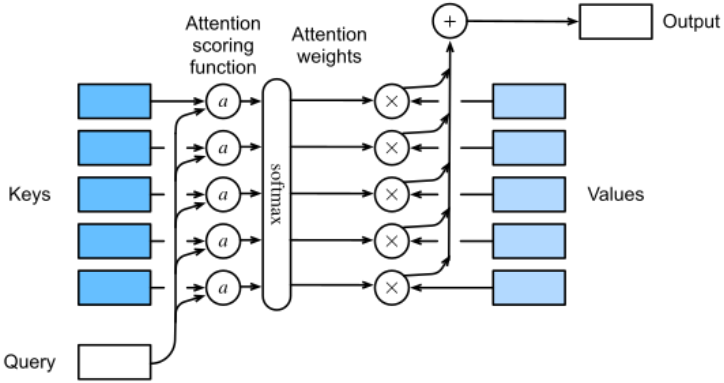
$$\text{Attention}(q) = \sum_{i=1}^{m} a(q, k_i) v_i$$

- $a(\cdot)$ are scalar "attention weights"; they give more weight ("pay more attention") to some items based on q and k_i.
- In a normal database query / dictionary, $a(q, k) = 1$ if $q = k$ and zero otherwise.
- in a transformer, this is generalized such that $a(\cdot) \geq 0$, $\sum a(\cdot) = 1$.
- achieved for any weighting function a_0 by a softmax operation:

$$a(q, k_i) = \text{softmax}(a_o(q, k_i)) = \frac{\exp(a_0(q, k_i))}{\sum_j \exp(a_o(q, k_j))}$$

  - $\uparrow$ differentiable and gradient never vanishes.

Scaled dot product attention



- let q and k be vectors with dimension d.
- scaled dot product attention:

$$a(q, k_i) = \text{softmax}(\frac{q \cdot k_i}{\sqrt{d}})$$

Self-Attention with word embeddings

- Consider a sequence of tokens with fixed length n_L, $\{w_1, ..., w_i, ..., w_{n_L}\}$
- We have (learnable) word embeddings $x_i = \omega_E w_i$ with dimension n_E, producing a sequence of vectors

$$\{x_1, ..., x_i, ..., x_{n_L}\}$$

Self-Attention with word embeddings

- Consider a sequence of tokens with fixed length n_L, $\{w_1, ..., w_i, ..., w_{n_L}\}$
- We have (learnable) word embeddings $x_i = \omega_E w_i$ with dimension n_E, producing a sequence of vectors

$$\{x_1, ..., x_i, ..., x_{n_L}\}$$

- A **self-attention layer** transforms $x_{1:n_L}$ into a second sequence $h_{1:n_L}$ with

$$h_i = \sum_{j=1}^{n_L} a(x_i, x_j) x_j$$

  - where $a(\cdot)$ is an attention function such that $a(\cdot) \geq 0$, $\sum a(\cdot) = 1$.
  - $\rightarrow$ each **context vector** h_i is a weighted average of the sequence of input word embeddings $x_{1:n_L}$.

Self-Attention with word embeddings

- Consider a sequence of tokens with fixed length n_L, $\{w_1, ..., w_i, ..., w_{n_L}\}$
- We have (learnable) word embeddings $x_i = \omega_E w_i$ with dimension n_E, producing a sequence of vectors

$$\{x_1, ..., x_i, ..., x_{n_L}\}$$

- A **self-attention layer** transforms $x_{1:n_L}$ into a second sequence $h_{1:n_L}$ with

$$h_i = \sum_{j=1}^{n_L} a(x_i, x_j) x_j$$

  - where $a(\cdot)$ is an attention function such that $a(\cdot) \geq 0$, $\sum a(\cdot) = 1$.
  - $\rightarrow$ each **context vector** h_i is a weighted average of the sequence of input word embeddings $x_{1:n_L}$.
  - if sequence length $n < n_L$, set $a_i = 0$ for all $i > n$.
- $h_{1:n_L}$ is flattened and piped to the network's hidden layers.

Basic Self-Attention

Setup:

1. Sequence of tokens $\{w_1, ..., w_i, ..., w_{n_L}\}$
2. Sequence of (trainable) word embedding vectors $\{x_1, ..., x_i, ..., x_{n_L}\}$
3. Sequence of attention-transformed context vectors $\{h_1, ..., h_i, ..., h_{n_L}\}$ with

$$h_i = \sum_{j=1}^{n_L} a(x_i, x_j) x_j$$

Basic Self-Attention

Setup:

1. Sequence of tokens $\{w_1, ..., w_i, ..., w_{n_L}\}$
2. Sequence of (trainable) word embedding vectors $\{x_1, ..., x_i, ..., x_{n_L}\}$
3. Sequence of attention-transformed context vectors $\{h_1, ..., h_i, ..., h_{n_L}\}$ with

$$h_i = \sum_{j=1}^{n_L} a(x_i, x_j) x_j$$

Basic self-attention uses scaled dot product attention:

$$a(x_i, x_j) = \text{softmax}(\frac{x_i \cdot x_j}{\sqrt{n_E}}) = \frac{\exp(\frac{x_i \cdot x_j}{\sqrt{n_E}})}{\sum_{k=1}^{n_L} \exp(\frac{x_i \cdot x_k}{\sqrt{n_E}})}$$

- the scaled dot-product $\frac{x_i \cdot x_j}{\sqrt{n_E}}$, normalized with softmax such that $\sum_j a(\cdot) = 1$.

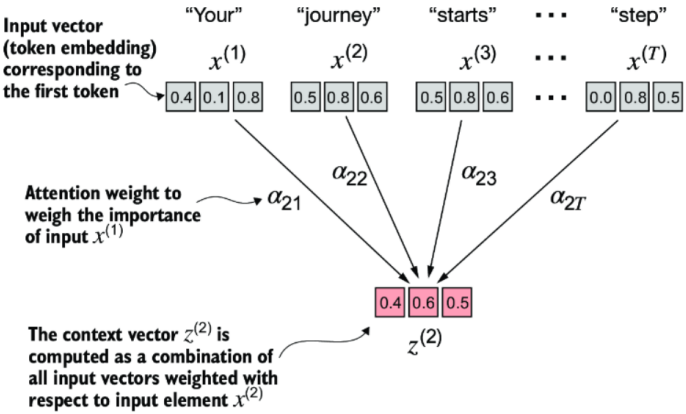


Figure 3.7 The goal of self-attention is to compute a context vector for each input element that combines information from all other input elements. In this example, we compute the context vector $z^{(2)}$. The importance or contribution of each input element for computing $z^{(2)}$ is determined by the attention weights a_{21} to a_{2T}. When computing $z^{(2)}$, the attention weights are calculated with respect to input element $x^{(2)}$ and all other inputs.

- The self-attention transformation

$$h_i = \sum_{j=1}^{n_L} a(x_i, x_j) x_j$$

with

$$a(x_i, x_j) = \text{softmax}(\frac{x_i \cdot x_j}{\sqrt{n_E}})$$

is a powerful architectural feature of transformers.

- The self-attention transformation

$$h_i = \sum_{j=1}^{n_L} a(x_i, x_j) x_j$$

with

$$a(x_i, x_j) = \text{softmax}(\frac{x_i \cdot x_j}{\sqrt{n_E}})$$

is a powerful architectural feature of transformers.

Note:

- **basic self-attention has no learnable parameters**.
  - self-attention works indirectly through the word embeddings (more next slide)
- **basic self-attention ignores word order.**

- The self-attention transformation

$$h_i = \sum_{j=1}^{n_L} a(x_i, x_j) x_j$$

with

$$a(x_i, x_j) = \text{softmax}(\frac{x_i \cdot x_j}{\sqrt{n_E}})$$

is a powerful architectural feature of transformers.

Note:

- **basic self-attention has no learnable parameters**.
  - self-attention works indirectly through the word embeddings (more next slide)
- **basic self-attention ignores word order.**

The big initial gain from transformers, relative to RNNs, came from basic self-attention.

- But the successful models (e.g. BERT, GPT) do add parameters and word order information to $a(\cdot)$.

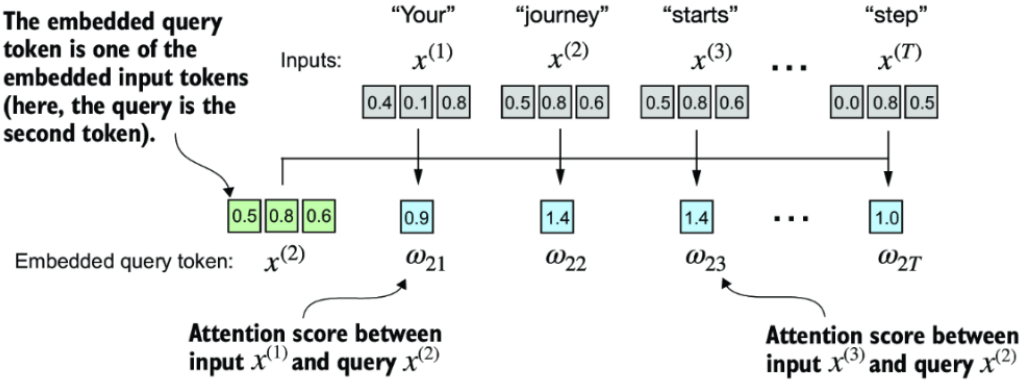


Figure 3.8 The overall goal is to illustrate the computation of the context vector $z^{(2)}$ using the second input element, $x^{(2)}$ as a query. This figure shows the first intermediate step, computing the attention scores w between the query $x^{(2)}$ and all other input elements as a dot product. (Note that the numbers are truncated to one digit after the decimal point to reduce visual clutter.)

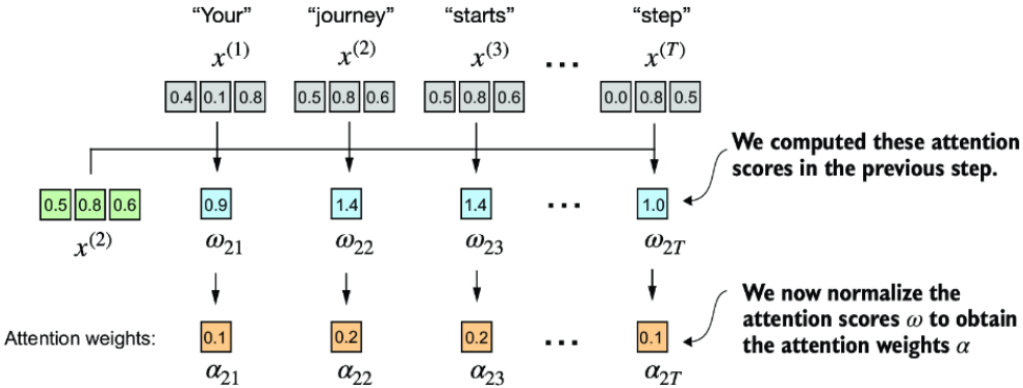


Figure 3.9 After computing the attention scores w_{21} to w_{2T} with respect to the input query $x^{(2)}$, the next step is to obtain the attention weights a_{21} to a_{2T} by normalizing the attention scores.

#1 The second input token is the query.

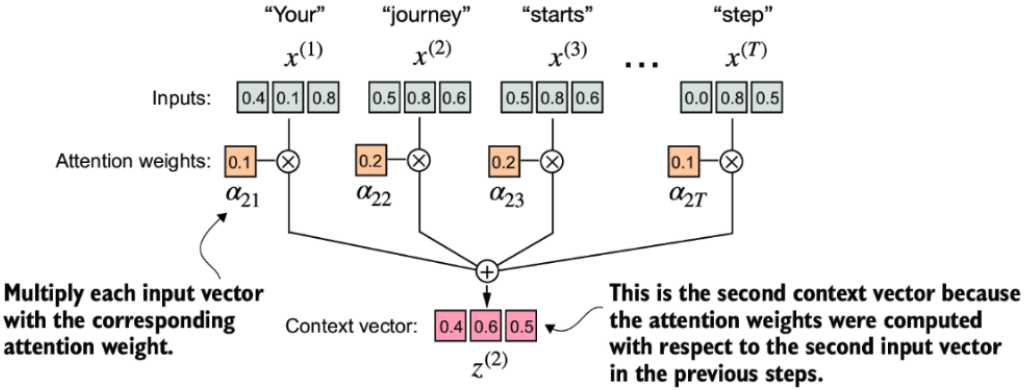


Figure 3.10 The final step, after calculating and normalizing the attention scores to obtain the attention weights for query $x^{(2)}$, is to compute the context vector $z^{(2)}$. This context vector is a combination of all input vectors $x^{(1)}$ to $x^{(T)}$ weighted by the attention weights.

	Your	journey	starts	with	one	step
Your	0.20	0.20	0.19	0.12	0.12	0.14
journey	0.13	0.23	0.23	0.12	0.10	0.15
starts	0.13	0.23	0.23	0.12	0.11	0.15
with	0.14	0.20	0.20	0.14	0.12	0.17
one	0.15	0.19	0.19	0.13	0.18	0.12
step	0.13	0.21	0.21	0.14	0.09	0.18

This row contains the attention weights (normalized attention scores) computed previously

Figure 3.11 The highlighted row shows the attention weights for the second input element as a query. Now we will generalize the computation to obtain all other attention weights. (Please note that the numbers in this figure are truncated to two digits after the decimal point to reduce visual clutter. The values in each row should add up to 1.0 or 100%.)

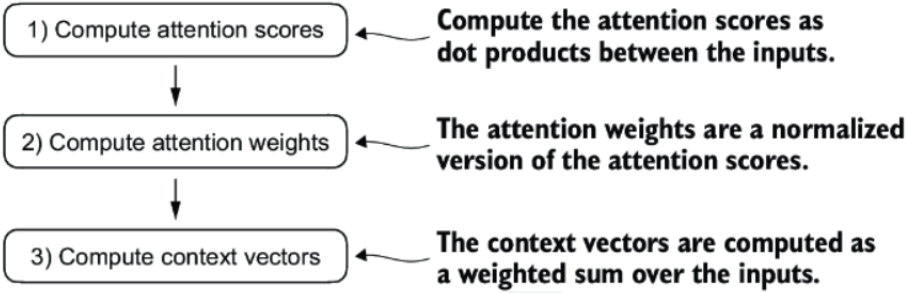


Figure 3.12 In step 1, we add an additional `for` loop to compute the dot products for all pairs of inputs.

Why self-attention works

- Consider a sentence

  the, cat, walks, on, the, street

  with embeddings

  $$\boldsymbol{x}_{\text{the}}, \boldsymbol{x}_{\text{cat}}, \boldsymbol{x}_{\text{walks}}, \boldsymbol{x}_{\text{on}}, \boldsymbol{x}_{\text{the}}, \boldsymbol{x}_{\text{street}}$$

- Feeding this sentence into the self-attention layer produces

  $$\boldsymbol{h}_{\text{the}}, \boldsymbol{h}_{\text{cat}}, \boldsymbol{h}_{\text{walks}}, \boldsymbol{h}_{\text{on}}, \boldsymbol{h}_{\text{the}}, \boldsymbol{h}_{\text{street}}$$

  where

  $$\boldsymbol{h}_i = a(x_i \cdot \boldsymbol{x}_{\text{the}})\boldsymbol{x}_{\text{the}} + a(x_i \cdot \boldsymbol{x}_{\text{cat}})\boldsymbol{x}_{\text{cat}} + \ldots + a(x_i \cdot \boldsymbol{x}_{\text{street}})\boldsymbol{x}_{\text{street}}$$

Why self-attention works

- Consider a sentence

  the, cat, walks, on, the, street

  with embeddings

  $$\boldsymbol{x}_{\text{the}}, \boldsymbol{x}_{\text{cat}}, \boldsymbol{x}_{\text{walks}}, \boldsymbol{x}_{\text{on}}, \boldsymbol{x}_{\text{the}}, \boldsymbol{x}_{\text{street}}$$

- Feeding this sentence into the self-attention layer produces

  $$\boldsymbol{h}_{\text{the}}, \boldsymbol{h}_{\text{cat}}, \boldsymbol{h}_{\text{walks}}, \boldsymbol{h}_{\text{on}}, \boldsymbol{h}_{\text{the}}, \boldsymbol{h}_{\text{street}}$$

  where

  $$\boldsymbol{h}_i = a(x_i \cdot \boldsymbol{x}_{\text{the}})\boldsymbol{x}_{\text{the}} + a(x_i \cdot \boldsymbol{x}_{\text{cat}})\boldsymbol{x}_{\text{cat}} + \ldots + a(x_i \cdot \boldsymbol{x}_{\text{street}})\boldsymbol{x}_{\text{street}}$$

Embedding layer will learn vectors $\boldsymbol{x}$ that tend to have **attention dot products** that contribute to the task at hand.

- For example, for most tasks, stopwords like "the" will not be helpful.
  - the learned embedding $\boldsymbol{x}_{\text{the}}$ will tend to have a low or negative dot product with more informative words.

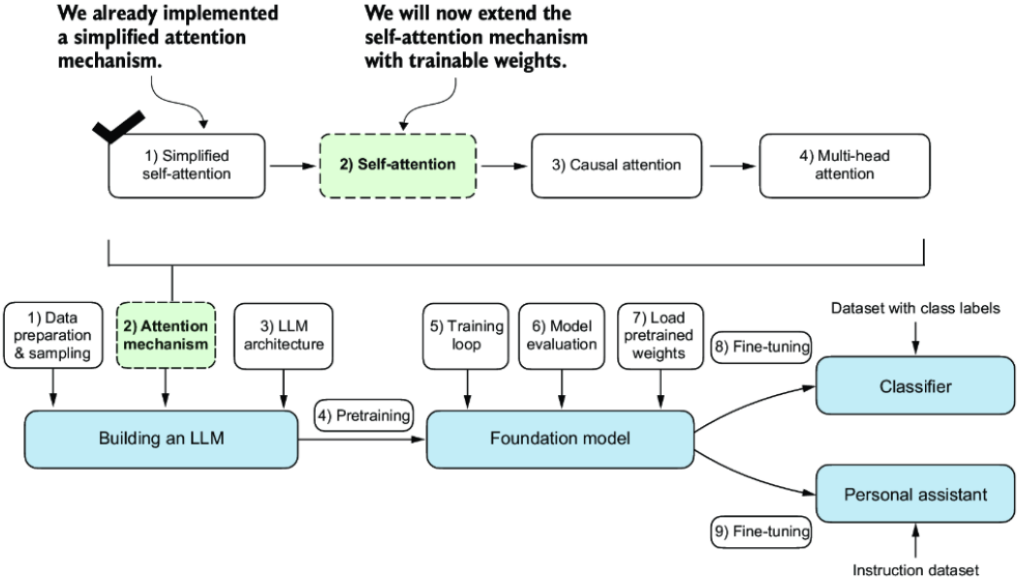


Figure 3.13 Previously, we coded a simplified attention mechanism to understand the basic mechanism behind attention mechanisms. Now, we add trainable weights to this attention mechanism. Later, we will extend this self-attention mechanism by adding a causal mask and multiple heads.

BERT (and RoBERTa)

- BERT = Bidirectional Encoder Representations from Transformers
  - RoBERTa = Robust BERT
- Architecture:
  - a stack of transformer blocks with a self-attention layer and an MLP.
  - The largest BERT model has 24 blocks, embedding dimension of 768, and 16 attention heads.
    $\approx$ 340M parameters to learn.

BERT (and RoBERTa)

- BERT = Bidirectional Encoder Representations from Transformers
  - RoBERTa = Robust BERT
- Architecture:
  - a stack of transformer blocks with a self-attention layer and an MLP.
  - The largest BERT model has 24 blocks, embedding dimension of 768, and 16 attention heads.
    $\approx$ 340M parameters to learn.
- Task: Masked language modeling:
  - 15% of words masked
  - if masked: replace with [MASK] 80% of the time, a random token 10% of the time, and left unchanged 10% of the time.
  - model has to predict the original word.

BERT (and RoBERTa)

- BERT = Bidirectional Encoder Representations from Transformers
  - RoBERTa = Robust BERT
- Architecture:
  - a stack of transformer blocks with a self-attention layer and an MLP.
  - The largest BERT model has 24 blocks, embedding dimension of 768, and 16 attention heads.
    $\approx$ 340M parameters to learn.
- Task: Masked language modeling:
  - 15% of words masked
  - if masked: replace with [MASK] 80% of the time, a random token 10% of the time, and left unchanged 10% of the time.
  - model has to predict the original word.
- Unlike GPT, BERT attention observes all tokens in the sequence, reads backwards and forwards (bidirectional).

BERT (and RoBERTa)

- BERT = Bidirectional Encoder Representations from Transformers
  - RoBERTa = Robust BERT
- Architecture:
  - a stack of transformer blocks with a self-attention layer and an MLP.
  - The largest BERT model has 24 blocks, embedding dimension of 768, and 16 attention heads.
    $\approx$ 340M parameters to learn.
- Task: Masked language modeling:
  - 15% of words masked
  - if masked: replace with [MASK] 80% of the time, a random token 10% of the time, and left unchanged 10% of the time.
  - model has to predict the original word.
- Unlike GPT, BERT attention observes all tokens in the sequence, reads backwards and forwards (bidirectional).
- Corpus:
  - 800M words from English books (modern work, from unpublished authors), by Zhu et al (2015).
  - 2.5B words of text from English Wikipedia articles (without markup).

'software engineers' Sentence		'petroleum engineers' Sentence	
Word	Prob.	Word	Prob.
it	0.08	energy	0.279
automotive	0.079	oil	0.27
technology	0.072	petroleum	0.088
healthcare	0.058	mining	0.035
insurance	0.053	defence	0.021
software	0.041	automotive	0.02
engineering	0.031	construction	0.017
public	0.03	gas	0.017
infrastructure	0.028	engineering	0.016
financial	0.028	water	0.012

Table 1: Predictions for Masked Words in Example Sentences

This tables displays masked word prediction probabilities for the two example sentences above. The training corpus for estimating these probabilities is English-language online job postings provided by Lightcast (formerly Emsi Burning Glass). The Transfomer model estimated for the task is DistilBERT (Sanh et al. 2020). See Hansen et al. (2023) for more details.